

This section documents the key architectural and technical design decisions made during the planning of the Teaching Vacancies Python rewrite, along with their rationale.

9.1. Choice of Python/Django for Rewrite

Decision: Rewrite the existing Ruby on Rails application using Python and the Django web framework. **Rationale: The primary driver is the organizational requirement to move to a centrally supported technology stack (Python).** Django is a mature, full-featured "batteries-included" framework that provides many parallels to Rails, potentially easing the conceptual transition for some aspects. It has a strong ecosystem, good documentation, and is suitable for building complex web applications like Teaching Vacancies. ** Python has a large talent pool, which can be beneficial for team building and long-term maintenance.

9.2. Custom User Model Strategy

Decision: Implement a single custom User model (extending `AbstractBaseUser` and `PermissionsMixin`) with a `user_type` field to differentiate roles (Jobseeker, Publisher, SupportUser). Common role-specific identifiers (like DSI OID or GOV.UK One Login ID) and a few role-specific fields are included directly on this User model for convenience and to avoid excessive joins for common data. More extensive, purely role-specific data (that doesn't fit all user types) will be in separate profile models linked via `OneToOneFields` (e.g., a future `JobseekerProfile` or `PublisherProfile` if needed beyond the fields on User). **Rationale: Provides a single point for authentication and core user identity using Django's built-in auth system.** The `user_type` field allows for easy role checking and differentiation in permissions and application logic. **Consolidating frequently accessed role-specific identifiers (like DSI OID) on the User model simplifies common queries and avoids mandatory `OneToOneField` joins for every user type for basic identification.** Balances the need for a unified user table with the ability to segregate more complex role-specific data into separate, optional profiles, adhering to Django best practices for user model extension.

9.3. Use of `django-allauth` for External IdPs

Decision: Utilize the `django-allauth` library to manage authentication flows with external Identity Providers (IdPs) like DfE Sign-In (DSI) and GOV.UK One Login (both via OpenID Connect/OAuth2). **Rationale: `django-allauth` is a comprehensive and well-maintained library that handles the complexities of OAuth/OIDC protocols, social account linking, and local**

account management (email/password for development/fallback). Reduces the development burden and security risks associated with implementing these complex authentication flows manually. ** Provides a consistent framework for handling multiple IdPs.

9.4. Database Choice - PostgreSQL with PostGIS

Decision: Continue using PostgreSQL as the relational database system and leverage the PostGIS extension for geospatial features. **Rationale: PostgreSQL is a robust, open-source, and feature-rich relational database, well-suited for complex applications. It was the database used by the previous Rails application.** PostGIS provides powerful geospatial querying capabilities essential for location-based vacancy searches and organisation mapping, a core feature of the platform. ** Django has excellent built-in support for PostgreSQL and GeoDjango for PostGIS.

9.5. Background Tasks - Celery with Redis

Decision: Use Celery for asynchronous task processing, with Redis as the message broker and results backend. **Rationale: Celery is a mature, feature-rich, and widely adopted distributed task queue system in the Python ecosystem. It supports scheduling (Celery Beat), retries, monitoring, and complex workflows.** Redis is a fast in-memory data store commonly used as a Celery broker due to its performance and simplicity. It can also be used for caching. ** This setup is well-suited for handling tasks like sending bulk emails (job alerts), data synchronization with external services (GIAS, DWP), and other long-running background operations.

9.6. API Framework - Django REST framework (DRF)

Decision: Use Django REST framework (DRF) for building any Web APIs, such as the one for external Applicant Tracking Systems (ATS). **Rationale: DRF is the de-facto standard for building RESTful APIs with Django. It provides a comprehensive toolkit including serializers, viewsets, authentication, permissions, and automatic API documentation generation.** Using DRF ensures consistency, maintainability, and adherence to best practices for API development.

9.7. Frontend Asset Bundling - esbuild

Decision: Utilize esbuild for frontend asset bundling (JavaScript, SCSS/CSS). **Rationale: esbuild is known for its exceptional speed, which can signif-**

icantly improve frontend build times, especially during development. The previous Rails application also used esbuild, so there is some existing familiarity and precedent within the project's context. ** While Webpack is more feature-rich, esbuild's speed and simplicity are advantageous for many common bundling tasks. If more complex JavaScript requirements arise (e.g., extensive code splitting, module federation beyond esbuild's capabilities), this decision could be revisited, but esbuild is a strong default.

9.8. UI Rendering - Django Templates with GOV.UK Design System Components

Decision: Primarily use server-side rendering with Django Templates. The frontend will strictly adhere to the GOV.UK Design System, implementing its components and patterns. **Rationale: Server-side rendering (SSR) with Django Templates is the traditional and simplest approach with Django, reducing complexity compared to a fully decoupled frontend (e.g., SPA with React/Vue). SSR is generally good for SEO and initial page load performance.** Adherence to the GOV.UK Design System is a mandatory constraint, ensuring consistency, accessibility, and a familiar user experience for users of UK government services. ** Reusable Django template tags or include snippets will be created to represent GOV.UK components, promoting consistency within the application and reducing code duplication.

9.9. Encryption - django-cryptography (or similar field-level encryption library)

Decision: Employ field-level encryption for sensitive Personally Identifiable Information (PII) at rest in the database. `django-cryptography` was identified as a candidate during implementation attempts, replacing older or less compatible libraries. **Rationale: Protects sensitive user data stored in the database, such as names, addresses, and details within job applications.** Field-level encryption allows specific model fields to be encrypted/decrypted transparently by the application, while still allowing non-sensitive fields to be queried normally. ** A specific library like `django-cryptography` (if compatible and maintained for Django 5.x) or a similar robust alternative is preferred over manual encryption implementation to ensure cryptographic best practices are followed. The exact library choice may require further validation for Django 5.x compatibility during full implementation.

9.10. Search - PostgreSQL Full-Text Search and GeoDjango

Decision: Initially leverage PostgreSQL's built-in full-text search (FTS) capabilities via `SearchVectorField` and GeoDjango for location-based searches using PostGIS. **Rationale:** PostgreSQL's FTS is powerful, integrated, and avoids the operational overhead of managing a separate search engine (like Elasticsearch or OpenSearch) for the initial rewrite, simplifying the architecture. GeoDjango provides robust geospatial query capabilities (e.g., radius searches, polygon intersections) directly within the Django ORM, integrating well with PostGIS. ** This approach meets the core search requirements (keyword, location, facets) effectively. If future performance or feature demands exceed what PostgreSQL can offer, migrating to a dedicated search engine can be considered as a later enhancement.

9.11. Notifications (In-App) - Custom Solution / django-notifications-hq (or similar)

Decision: For in-app notifications (e.g., dashboard alerts for publishers about new applications, distinct from email notifications via GOV.UK Notify), a solution will be implemented. This might be a custom model-based solution or a library like `django-notifications-hq`. The Rails app used the `noticed` gem. **Rationale:** The `noticed` gem in Rails provided a flexible way to manage different types of notifications. A Django equivalent is needed for user-facing alerts within the application. A custom solution would involve creating `Notification` models linked to users and relevant trigger events/objects. ** A library like `django-notifications-hq` offers pre-built components for common notification patterns (unread status, actor/target/action object, rendering), potentially speeding up development. The choice will depend on the complexity of notification requirements versus the overhead of integrating a library.

9.12. Auditing - django-reversion / django-simple-history

Decision: Implement model versioning/auditing using a library like `django-reversion` or `django-simple-history`. The Rails app used `papertrail`. **Rationale:** Tracking changes to key models (especially `Vacancy` and `JobApplication`) is important for support, debugging, and understanding data history. Libraries like `django-reversion` or `django-simple-history` provide robust mechanisms for storing model versions, tracking who made changes, and reverting to previous states if necessary. This is more reliable and feature-rich than a manual implementation. The choice between them will depend on specific features needed (e.g., ease of admin integration, granularity of diffs).

9.13. Modular Django App Structure

Decision: Organize the Django project into distinct applications, each representing a major domain or functional area (e.g., `users`, `organisations`, `vacancies`, `job_applications`, `jobseekers`, `publishers`, `core`, `notifications`, `api`). **Rationale:** Promotes separation of concerns, making the code-base easier to understand, navigate, and maintain. Allows different developers or teams to work on different parts of the application with fewer conflicts. **Improves testability by allowing tests to be focused on specific app functionalities.** Follows standard Django best practices for project organization.